Version lineage: keep every copy up, and say which one this is

Anyone can compare versions side by side, and nobody is ever confused about where they are.

Kind Pattern | Changed 2026-11-04 | For Anyone who moves or rehearses a site

You are here: drasticstatic lab › `my-template` › `walkthroughs` › `VERSION-LINEAGE.md` · public · `main` is the working branch

TL;DR

1. When a site moves or rehearses, **don't hide the other copies from search (`noindex`) and don't take them down.**
2. **Every copy that isn't live** greets visitors once with a short note: *"You're on a rehearsal stage"* or *"You're looking at an earlier version"*. It links to **every newer version** and, below that, to **every earlier version** (the time machine).
3. **The live site shows nothing** and never links back.
4. **A canonical link** tells search engines which copy is the real one.

First built for I Am One Self (The Holy Earth Foundation), 2026-10-25. It's the same idea as keeping old test publishes up beside the live site, made visitor-friendly.

The rules

Rule	Why
The live site has no note and no links back	Visitors on the real site are never distracted by its history
Each other copy lists the newer stops first, then the time machine: the earlier stops, nearest first	Forward is the main path: an earlier version points ahead, and a rehearsal points to live. Stepping back is a choice, so anyone can compare *then*, *Genesis day* and *next* side by side. (The first version listed only newer stops. Christopher asked for the time machine on move day, 2026-10-25, so the rehearsal could "play back in reverse" too)
The note stays until the visitor confirms, then never again **in that browser, for that copy**	Honest without nagging. Remembered per copy, so confirming the rehearsal doesn't silence the old version
A small ribbon in the corner reopens it	The truth is always one tap away
`<link rel="canonical">` points to live on every other copy	Search engines credit the live site, while the copies stay findable and useful
No `noindex`	Old versions are part of the story. They teach, and they prove where the work came from
Stops that don't exist yet are listed but closed (`open: false`)	The list is ready in advance; a move only flips switches

The list (one file, oldest first)

```ts
// src/data/lineage.ts
export type LineageKind = "earlier" | "rehearsal" | "live";

export interface LineageStop {
  id: string;      // what each deploy lane names itself
  label: string;   // "The Genesis rehearsal"
  note: string;    // one line under the label
  url: string;
  kind: LineageKind;
  open: boolean;   // false = not built yet; flip it when it is
}

export const LINEAGE: LineageStop[] = [
  { id: "lab-original",   kind: "earlier",   open: false, /* opens at the move */ label: "The first lab version", ... },
  { id: "lab-genesis",    kind: "rehearsal", open: true,  label: "The Genesis rehearsal", ... },
  { id: "thef-rehearsal", kind: "rehearsal", open: false, label: "The foundation's rehearsal stage", ... },
  { id: "live",           kind: "live",      open: true,  label: "The live site", ... },
];

export const LINEAGE_SELF = process.env.NEXT_PUBLIC_LINEAGE_SELF ?? "";

/** This build's stop, unless it is the live site (or no lane set it). */
export const lineageSelf = () => LINEAGE.find((s) => s.id === LINEAGE_SELF && s.kind !== "live");

/** Every stop newer than this one that exists today. */
export function newerStops() {
  const i = LINEAGE.findIndex((s) => s.id === LINEAGE_SELF);
  return i < 0 ? [] : LINEAGE.slice(i + 1).filter((s) => s.open);
}

/** The time machine: every stop before this one that exists today, nearest first. */
export function olderStops() {
  const i = LINEAGE.findIndex((s) => s.id === LINEAGE_SELF);
  return i < 0 ? [] : LINEAGE.slice(0, i).filter((s) => s.open).reverse();
}
```

At the move, `lab-original` opens too: once it's history, the rehearsals' time machines can reach it.

Local builds and the live site set no `LINEAGE_SELF`, so they render nothing. Only a lane that names itself shows the note.

The note (the parts that matter)

- **A native `<dialog>` opened with `showModal()`.** You get focus trapping, a backdrop and screen-reader semantics for free.
- **Remember the confirmation in `localStorage`,** keyed to *this copy's* id, inside `try/catch`. In private mode it simply greets them again next time. That's fine.
- **Decide after hydration** (`setTimeout(…, 0)`). `localStorage` isn't readable during static rendering.
- **The Escape gotcha:** Chrome lets Escape close a dialog that opened without a click, even if you `preventDefault()` the `cancel` event. So in `onClose`, if the visitor hasn't confirmed, call `showModal()` again on the next tick. The note can be dismissed only by choosing.

- **Two choices, both honest:** *"Take me to the live site"* and *"I understand, let me look around"*.
- **The time machine:**
  - It sits under *"Newer versions"*, headed *"Earlier versions"* with the line *"Step back in time to see how it grew."*
  - Each earlier stop has a rewind (History) icon and a **dashed** edge, so the past reads as memories.
  - The ribbon says so too: *"Rehearsal · not live · all versions"*, and the foot of the note reads *"The ribbon in the corner brings this note back, with every version."*
- **Keep it out of search results** (`data-pagefind-ignore`).
- **Let other welcomes wait for it.** On I Am One Self, the arrival splash listens for a `lineage:ack` event, so two dialogs never stack.
- **Both languages:** every label and note goes through the site's translation table.

Wiring it into the deploy lanes

Each lane names its stop and its canonical target at build time:

```
# .github/workflows/lanes.yml (build step env)
NEXT_PUBLIC_LINEAGE_SELF: ${{ github.ref_name == 'main' && 'live' || vars.REHEARSAL_LINEAGE }}
NEXT_PUBLIC_CANONICAL_URL: ${{ github.ref_name != 'main' && format('{0}{1}', vars.LIVE_SITE_URL,
vars.LIVE_BASE_PATH) || '' }}
```

- **Live builds** are `live` , with no canonical override. They render nothing.
- **The rehearsal** sets the repository variable `REHEARSAL_LINEAGE` (e.g. `lab-genesis` , later `thef-rehearsal`).
- **An old copy** that keeps building from its own workflow sets its own id (`lab-original`) the same way.

The page's metadata adds `<link rel="canonical" href={NEXT_PUBLIC_CANONICAL_URL}>` whenever it's set.

At a move (how the list changes)

Moment	Flip
Before: testing the new home in a lab	`lab-genesis` open; the old site is still live, so it shows nothing
At the move: the domain points to the new home	Open the new rehearsal stop. Open the old copy as `earlier`, and give its build `LINEAGE_SELF=lab-original` so it greets visitors and points forward
After	Nothing to maintain. New rehearsals are added as new stops, and old ones stay as history

> ***Gotcha for a new organization:*** *GitHub turns deploy keys **off** for new orgs. The lanes publish with one write deploy key per public repo, so an owner turns them on before the move: Org settings → Member privileges → Deploy keys → **Enabled** → Save. Each key can write to its one repository only. A GitHub App is the alternative if you ever want finer control. Found at The Holy Earth Foundation's move (2026-10-25), where the setup script stopped cleanly at its first key.*

> ***Gotcha: new repos may come up with Actions off.*** *The repo's Actions tab says "Workflows aren't being run on this repository", even though the API reports Actions as enabled.*
>
> - ***Symptom:*** *pushes and merged pull requests publish nothing, while manual (`workflow_dispatch`) runs work.*
> - ***Fix:*** *an owner clicks **Enable Actions on this repository**, once per new workshop repo. Do it before the first Promote.*

> ***Gotcha: an empty value is false in GitHub expressions.*** *A custom domain has an empty* `LIVE_BASE_PATH`*, so* `main && vars.LIVE_BASE_PATH || vars.REHEARSAL_BASE_PATH` *falls through and builds the live site for the rehearsal's subpath: the pages load, but every style 404s. Choose between JSON strings instead:*
>
> ```
> NEXT_PUBLIC_BASE_PATH: ${{ fromJSON(github.ref_name == 'main' && toJSON(vars.LIVE_BASE_PATH) || toJSON(vars.REHEARSAL_BASE_PATH)) }}
> ```
>
> *Check any* `a && b || c` *whose* `b` *can be empty. (* `… && vars.LIVE_CNAME || ''` *is safe, because its fallback is empty anyway.)* ***Verify a live build by fetching a real asset:*** *take a CSS path from the page's HTML and request it, rather than checking only that the page itself returns 200.*

Twins: keeping the copies in step after a move

Once a site has copies in two places (a new home and the lab it came from), a fix in one may belong in the other. Write down what each copy receives, in the workshop's `AGENTS.md`:

Copy	Receives
The new home's workshop	Everything. It's the source of truth
The lab's Genesis lane	Shared features (the lineage note and time machine, welcomes, trails) and security or dependency fixes. Not day-to-day content: it stays "Genesis day"
The earlier version	Security or dependency fixes and forward lineage links only. Its words stay as they were

The chore:

- Add a **Twin check** box to the pull request template.
- Run a read-only script that lists, per site, the files that differ between the workshop and the Genesis lane, plus the `npm audit` count of every copy and each repo's open Dependabot alerts. The Holy Earth Foundation's is `twin-check.sh`.
- Mirror **file for file**, only after confirming the twin's file still matches the last mirrored version.
- Bring the earlier versions' Dependabot pull requests together into one tested pull request (build with the publisher's own settings, then check canonicals and styles), and close major tooling jumps on a frozen copy.
- Point weekly version updates at `preview` (`target-branch`) so they rehearse first. Security fixes always open against `main`: retarget them to `preview` before merging.

Using it elsewhere

- **Next.js sites:** copy `lineage.ts`, `LineageModal.tsx` and `lineage.css` from I Am One Self, mount the modal in the root layout, and add the two env lines to your lanes.
- **Kit sites (AstroLog, portals):** the same rules suit a small plain-JavaScript version that reads its id from a `<meta>` tag. It's on the kit 1.5 wish list.
- **dApps with demo deployments:** the same note replaces "this is a demo" banners. List the production app as `live`.

Suggest a change

Noticed something? Open an issue or a pull request against `main`, changing this page; a steward reads it and a person with the role merges, with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge, signed as the attribution spec says. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Part of the walkthroughs. Icons: Lucide (ISC).